

NEURAL VS. CLASSICAL DECODERS FOR THE SURFACE CODE: FROM SIMULATION TO IBM QUANTUM HARDWARE

Esranur Aygün

*Department of Computer Engineering
Bahçeşehir University
Istanbul, Türkiye
esranur.aygun@bahcesehir.edu.tr*

Abstract — Quantum computers promise incredible speed; however, they are fundamentally fragile. Today’s physical qubits make an error roughly once every 1000 operations. Running useful algorithms requires error rate to be around 1 in a trillion. Since quantum hardware is not up to such standard yet, Quantum Error Correction (QEC) via surface code is one of the leading approaches to solve this problem. This study presents direct comparison of classical and neural decoders on distance-3 rotated surface code. Simulated and real hardware noise is evaluated on Minimum-Weight Perfect Matching (MWPM) algorithm, a Convolutional Neural Network (CNN), a Transformer, and hybrid two-stage Transformer model. Findings show that hybrid architecture achieves Logical Error Rate (LER) of 12.65% under simulated asymmetric noise, outperforming MWPM’s 19.90%. Validation using 1,000-shot syndrome data from an IBM quantum processor demonstrates an LER of 49.70% for the Hybrid model compared to 50.50% for MWPM. Results show that neural decoding solutions open a path for fault tolerance quantum computing that are hardware agnostic.

Index Term — Quantum Error Correction, Surface Code, Neural Decoder, Minimum-Weight Perfect Matching, Transformer

I. INTRODUCTION

Quantum computers offer rapid solutions across various fields. Yet, their practical use cases are restricted by hardware fragility. Current physical error rates are around 0.1% to 1% per operation. This is quite high for most quantum algorithms, which need these rates closer to 10^{-12} . Quantum Error Correction focuses on reducing this gap by combining many fragile physical qubits together to act as one logical qubit [1].

Among the QEC architectures, surface code is one of the leading decoding algorithms, due to how it can handle higher rate of baseline error, scale in size, and relies on localized connections [2]. In a surface code, data and ancilla qubits are arranged in a 2-dimensional $d \times d$ grid to run syndrome extraction rounds to find errors without measuring the actual encoded information, which would make the system collapse to superposition. A decoder algorithm reads these measurements to fix errors; however traditional ones cannot keep up with scaled and noisy quantum information.

A common traditional decoder for surface code is Minimum-Weight Perfect Matching (MWPM), but it tends to fall short because it can’t adapt well to shifting real hardware noise. Google DeepMind’s AlphaQubit proposed a neural network trained in two stages, first on simulated noise and then fine-tuned on actual machine data, can learn to correct errors directly from the

system [3]. Inspired by this, this paper compares four decoders on a distance-3 rotated surface code: classical MWPM, a CNN, a standard Transformer, and a two-stage Hybrid Transformer. These are tested on simulated symmetric/asymmetric noise, and real IBM hardware data, and found that this method adapts different noise data and beats both MWPM and other networks.

The code for this papers experiments and model architectures can be found in “<https://github.com/fukichime/quantum-noise-decoder-comparison.git>”

II. RELATED WORK

A. *Hardware Race in Quantum*

The decoding problem is universally investigated in the quantum industry. IBM is developing superconducting qubit processors and achieved high-threshold for fault-tolerant quantum memory using qLDPC codes [4]. Researchers at Harvard University have developed programmable logical quantum processors using reconfigurable neutral-atom arrays [5]. Microsoft is exploring topological quantum computation via Majorana-based qubits [6]. Each of these platforms generate complex, biased noises that classical decoders struggle to handle, showing the need for approaches that directly learn from data, not hardware.

B. *Classical Baseline*

Despite the variety of hardware options, Minimum-Weight Perfect Matching remains as classical standard decoder. Higgott and Gidney's optimized PyMatching library made MWPM the practical baseline for most surface code experiments [7]. MWPM is a traditional graph-based algorithm that connects syndrome nodes to find the high likely error pattern using the shortest path. However, it performs poorly when noise is non-uniform. A recent study showed that MWPM performs well under ideal noise, starts losing accuracy when noise is biased or non-identical [8]. The degrading occurs because MWPM solves X and Z errors on *separate* graphs, meaning it fails to understand errors are correlated or not [9]. This limitation prevents the classical method from producing meaningful results on real hardware.

C. *Neural Decoders*

Researchers are shifting towards machine learning to overcome these limitations of classical methods. Especially, neural decoders perform well at learning complex error patterns from real hardware data. Recent work by Wang et al. studied Transformer-QEC to decode across different code distances [10]. Google DeepMind's AlphaQubit achieved top-tier decoding accuracy by outperforming MWPM by pre-training on simulated noise and fine-tuning on real experimental data [3].

As the quantum computers grow larger, advantage of neural decoders will be more prominent. Adapt to this scalability, AlphaQubit 2 extended the work by getting near-perfect error correction at real-time speeds under 1 μ s per cycle [11]. Meanwhile, Hall, Gicev, and Usman

showed these methods to work in the real world by using artificial neural decoders on actual IBM quantum computers [12]. These works form the foundation for this paper to improve upon neural network solutions and prove their reliability.

III. METHODOLOGY

A. Surface Code Circuit and Noise Models

Distance-3 rotated surface code based on stabilizer formalism is implemented to evaluate the decoders [1]. The quantum circuit contains total of 17 physical qubits: 9 data qubits to hold logical information and 8 ancilla or helper qubits. The circuit executes five consecutive syndrome extraction rounds.

In each round, Hadamard gates applied to create superposition, and CNOT gates to check the neighboring data qubits for mismatches. The ancilla qubits record the parity measurement without looking at the actual data. After 5 rounds, the data qubits are measured to get our ground-truth label, and the 40-bit error pattern ($5 \text{ rounds} \times 8 \text{ ancillas}$) is extracted from syndrome.

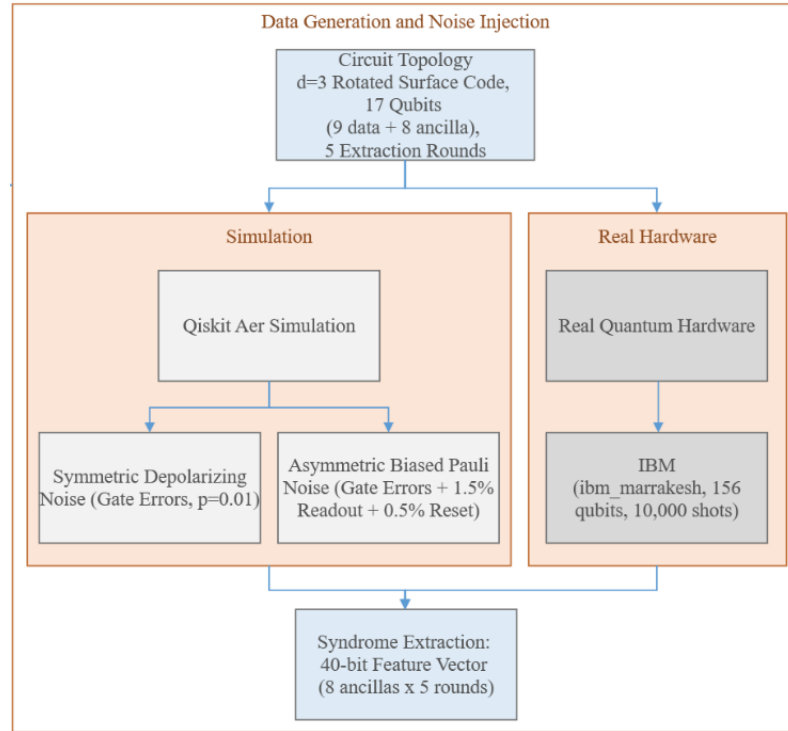


Figure 1. Experimental pipeline for quantum error correction data generation.

The complete noise injection and data generation pipeline is illustrated in Figure 1. All simulations were developed using Python and executed via the Qiskit AerSimulator. Two distinct simulated datasets containing 10,000 shots each were generated.

The first dataset uses a symmetric depolarizing noise model to simulate a perfect, theoretical setting where bit-flips (X-errors) and phase-flips (Z-errors) occur at the exact same rate. This dataset is used to pre-train the neural networks, to teach the models the basic mathematical logic and geometry of the surface code.

The second dataset uses an asymmetric, biased Pauli noise model to mimic real quantum hardware. In actual physical chips, qubits frequently lose energy and relax back to zero rather than flipping randomly. To replicate hardware, a 1.5% measurement error and a 0.5% reset error heavily biased toward X-flips are injected. This biased dataset is used for fine-tuning, teaching the network how to manage real-world, unpredictable noise.

Furthermore, to validate models against authentic physical noise, same distance-3 circuit executed on an actual IBM superconducting quantum processor (ibm_marrakesh), collecting 1,000 shots of real-world hardware data [12]. Simulators are limited because they cannot model physical effects like crosstalk, where signals leak between neighboring qubits, or energy leakage from qubits losing their quantum state. Running on real hardware presented a challenge: raw physical readouts had to be formatted into clean 40-bit tensors. IBM's SamplerV2 primitive is used to return true raw probability distributions rather than just statistical averages. Whether simulated or real, the output formats are identical: a 40-bit syndrome feature vector paired with a 1-bit logical outcome label.

B. Decoder Architecture and Training pipeline

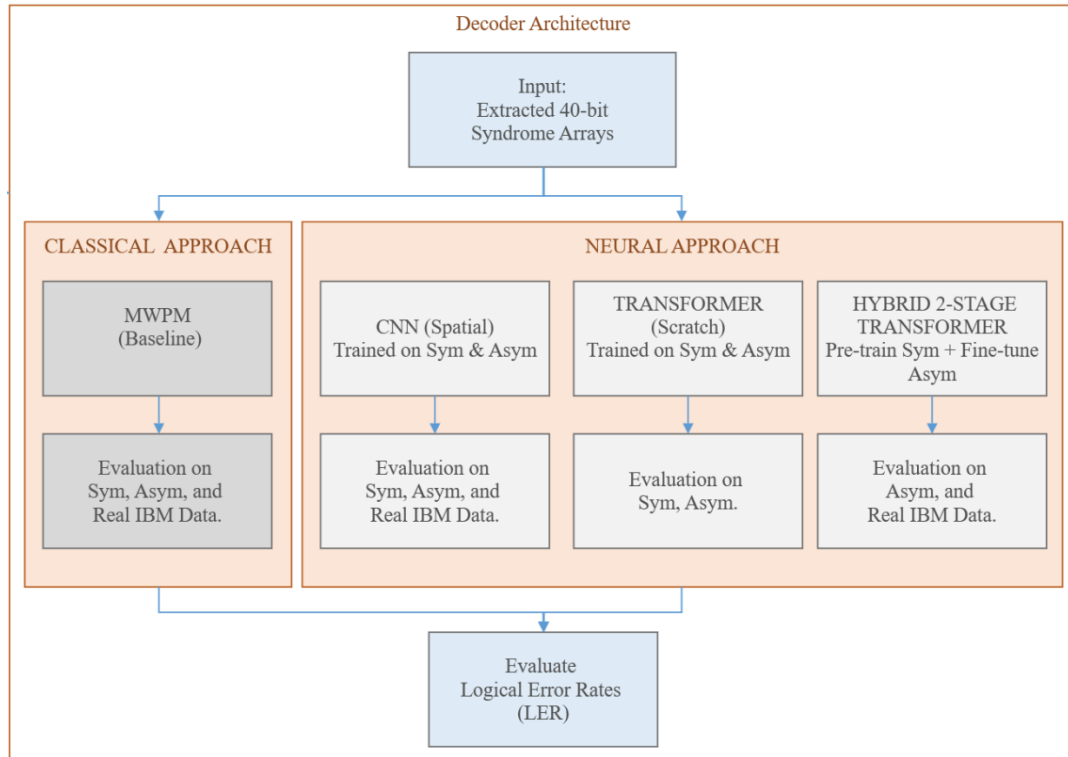


Figure 2. Training and evaluation pipeline for the decoders.

Once the syndrome data is extracted, it is passed into four decoding algorithms to see the most effective method for correcting errors, as detailed in Figure 2.

First, Minimum-Weight Perfect Matching was applied as classical baseline. The PyMatching library is used to translate the syndrome rounds into a mathematical detection graph to find the most probable error chain [7]. MWPM doesn't require training, is rule-based, and gives accuracy threshold.

Same datasets processed through three neural network architectures:

- 1) ***Convolutional Neural Network (CNN)***: Trained directly on the respective noise datasets to evaluate how well spatial convolutions can identify local hardware biases.
- 2) ***Transformer (Trained from Scratch)***: A self-attention neural network trained completely from scratch on both symmetric and asymmetric noise data. This serves as a control strategy to evaluate how efficiently a blank model can learn complex noise directly.
- 3) ***Hybrid Two-Stage Transformer***: This proposed model is initially pre-trained on the symmetric noise dataset to learn the fundamental topological geometry of the surface code, then fine-tuned on the complex asymmetric noise to adapt to physical noise patterns

All four approaches evaluated against both the simulated and the real physical data.

IV. RESULTS AND DISCUSSION

For decoder evaluation, their performance first analyzed on the simulated distance-3 surface code, then followed by validation on the real IBM quantum processor. The performance breakdown is provided in Table I.

A. Simulated Performance

Under ideal symmetric noise, the neural networks outperformed the classical baseline. The Transformer trained from scratch achieved a Logical Error Rate (LER) of 7.25%, reducing MWPM's 15.00% error rate by more than half. Yet, real hardware does not produce ideal noise.

When subjected to asymmetric Pauli noise, MWPM suffered a significant performance drop, with its error rate surging to 19.90%. Because MWPM uses on fixed mathematical graphs, it cannot adapt to correlated device noise. In contrast, the neural networks demonstrated strong adaptability. The CNN reduced the error to 15.15% by learning local spatial patterns [12]. The Transformer trained from scratch achieved 13.05%, while the Hybrid 2-Stage Transformer delivered the best result at 12.65% [3]. Initially capturing geometric structure, then adjusting to hardware effects, the Hybrid model established itself as the best-performing decoder in simulation.

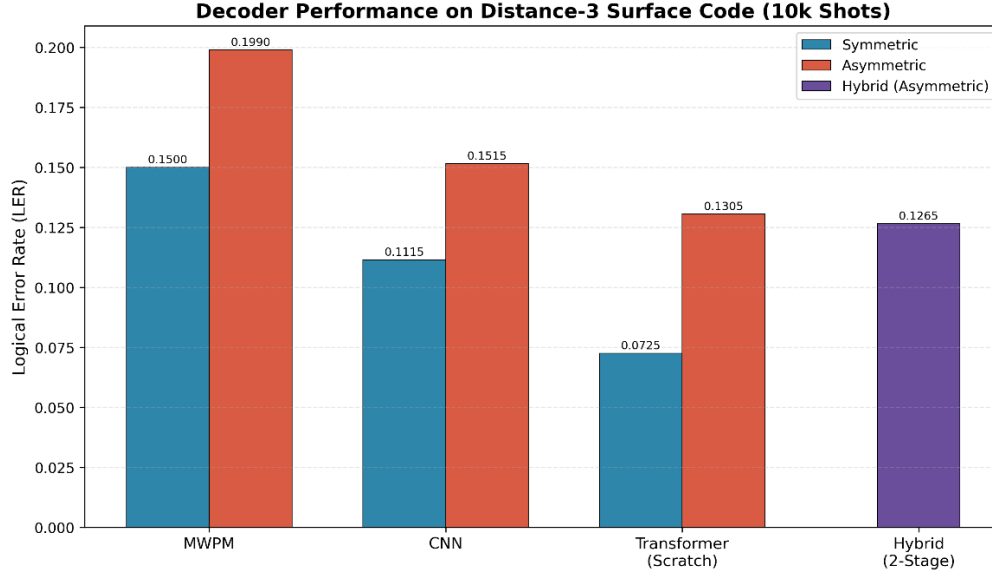


Figure 3. Simulated Logical Error Rates (LER) for the distance-3 surface code across 10,000 shots.

B. Real Hardware Validation

Simulators cannot model physical crosstalk or energy leakage, which is why we tested the models on 1,000 shots from the `ibm_marrakesh` processor. As shown in Table I, real hardware pushed all error rates near 50%.

As seen in Figure 4, the CNN performed worse than MWPM on real hardware (51.40% vs. 50.50%). This reveals that simply using a neural network is not enough. The CNN learned simulated spatial patterns but failed to generalize them to physical noise. Conversely, the Hybrid 2-Stage Transformer successfully transferred its geometric knowledge to real hardware, achieving an LER of 49.70%. While a 0.80% improvement over MWPM seems modest, fractions of a percent are often decisive in fault-tolerant algorithms. This hardware validation proves that two-phase training enables Transformers to handle unpredictable physical noise.

Table 1. Final Decoder Evaluation: Simulation Vs. Real Hardware (Distance-3)

Decoder Architecture	Symmetric Noise	Asymmetric Noise	IBM Hardware
MWPM (Classical)	15.00%	19.90%	50.50%
CNN	11.15%	15.15%	51.40%
Transformer (Scratch)	7.25%	13.05%	—
Hybrid 2-Stage	—	12.65%	49.70%

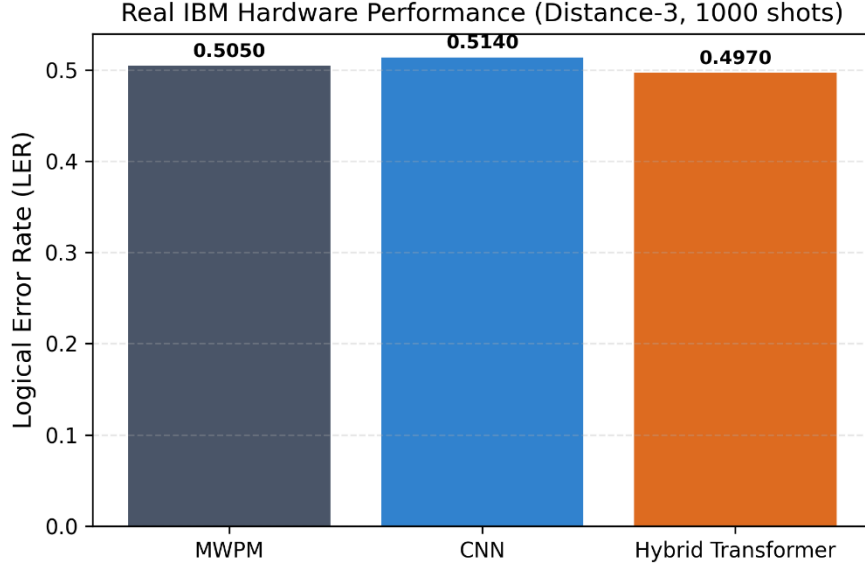


Figure 4. Real IBM Hardware Performance (Distance-3, 1000 shots)

V. CONCLUSION

In this work, classical and neural decoders compared for the surface code under simulated and real hardware noise. results confirm that MWPM, the industry standard, fails under non-uniform noise. Its error rate increased by nearly 5% when moving from ideal symmetric noise to realistic asymmetric noise, highlighting its lack of adaptability.

Neural networks proved more adaptable. The CNN reduced the asymmetric noise error rate to 15.15%, and the Transformer trained from scratch achieved 13.05%. The Hybrid 2-Stage Transformer delivered the best simulated performance at 12.65%, a 7.25% improvement over MWPM. This two-stage approach proved highly effective [3].

The most significant validation came from testing on the `ibm_marrakesh` processor. Real hardware pushed all decoders to roughly 50% error rates due to crosstalk and leakage. The CNN failed to generalize, performing worse than MWPM. However, the Hybrid Transformer maintained the lowest error rate at 49.70%, outperforming MWPM's 50.50%. This confirms that phased training enables neural decoders to transfer geometric knowledge to real hardware, making them more robust under physical noise.

These findings confirm that phased training improves neural decoder performance on real hardware. The Hybrid Transformer adapted to physical noise patterns that MWPM could not handle, achieving the lowest error rate on the IBM processor. This work demonstrates that neural decoding is a practical direction for future quantum systems where classical methods fall short.

Future work can explore larger surface codes that would require more training data to handle the expanded error space and faster decoding times would also need optimized models and hardware support.

REFERENCES

- [1] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, "Topological quantum memory," *J. Math. Phys.*, vol. 43, no. 9, pp. 4452–4505, Sep. 2002.
- [2] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Phys. Rev. A*, vol. 86, no. 3, p. 032324, Sep. 2012.
- [3] J. Bausch et al., "Learning high-accuracy error decoding for quantum processors," *Nature*, vol. 635, pp. 834–840, Nov. 2024.
- [4] S. Bravyi et al., "High-threshold and low-overhead fault-tolerant quantum memory," *Nature*, vol. 627, no. 8005, pp. 778–782, Mar. 2024.
- [5] D. Bluvstein et al., "Logical quantum processor based on reconfigurable atom arrays," *Nature*, vol. 626, no. 7997, pp. 58–65, Feb. 2024.
- [6] Microsoft Azure Quantum, "Interferometric single-shot parity measurement in InAs–Al hybrid devices," *Nature*, vol. 638, p. 651, Feb. 2025.
- [7] O. Higgott and C. Gidney, "Sparse Blossom: correcting a million errors per core second with minimum-weight matching," *Quantum*, vol. 9, p. 1600, Jan. 2025.
- [8] A. deMarti iOlius, J. E. Martinez, P. Fuentes, and P. M. Crespo, "Performance of surface codes in realistic quantum hardware," *Phys. Rev. A*, vol. 107, no. 6, p. 062401, Jun. 2023.
- [9] A. deMarti iOlius, J. E. Martinez, P. Fuentes, and P. M. Crespo, "Performance enhancement of surface codes via recursive minimum-weight perfect-match decoding," *Phys. Rev. A*, vol. 110, no. 1, p. 012426, Jul. 2024.
- [10] H. Wang et al., "Transformer-QEC: Quantum Error Correction Code Decoding with Transferable Transformers," *arXiv:2311.16082*, Nov. 2023.
- [11] A. W. Senior et al., "A scalable and real-time neural decoder for topological quantum codes," *arXiv:2512.07737*, Dec. 2025.
- [12] B. Hall, S. Gicev, and M. Usman, "Artificial neural network syndrome decoding on IBM quantum processors," *Phys. Rev. Res.*, vol. 6, p. L032004, Jul. 2024.